BAC Maps: A Web-Based Campus Navigation System for Botswana Accountancy College

by

Kaelo Kealeboga Selome ([**INSERT student ID**])
Monyana Bianca Kedukaetswe ([**INSERT student ID**])
Larona Mothame ([**INSERT student ID**])
Katlo Tutwane ([**INSERT student ID**])
Abigail Mototegi ([**INSERT student ID**])

A report submitted in partial fulfilment of the requirements for the Research and Innovation module

Botswana Accountancy College

Supervisor: [**INSERT supervisor's name, or delete this line**]

[**INSERT submission date**]

Declaration

We, the undersigned, declare that this report is the result of our independent work and that all sources used have been acknowledged. This work has not previously been submitted to any institution for academic credit. It is submitted in partial fulfilment of the requirements for the Research and Innovation module at Botswana Accountancy College. (Kaelo Kealeboga Selome, Monyana Bianca Kedukaetswe, Larona Mothame, Katlo Tutwane and Abigail Mototegi)

Signatures: [**INSERT signatures and date**]

Acknowledgements

[INSERT acknowledgements: for example the supervisor, the staff who supplied the campus drawings and timetable, and anyone who walked the campus routes]

Abstract

New students and visitors at Botswana Accountancy College (BAC) find rooms by asking for directions, because no map of the campus shows the rooms inside its buildings or how to walk to them. This report presents BAC Maps, a web application that shows the campus, its two-storey Main Building and its 40 mapped rooms, and gives walking directions that stay outside walls and use the stairs. The work followed an iterative prototype design. The campus was traced in Google Earth, ten walks were measured on site with GPS, and two campus drawings were aligned to the traces with a thin-plate spline (median error 1.6 m). Routes are computed with the A* search algorithm over a 0.5 m grid outdoors and a graph of corridors, doors and stairs indoors. This version made five changes: the interface was joined to the finished map and route engine, each student's class-group timetable was added to the profile, routes were kept clear of the directions panel, the interface was rebuilt in React, and ten defects found in testing were fixed. Of the 20 functional requirements in the revised catalogue, 16 were met and four partly met; the camera-based door recognition first planned was descoped. On 24 October 2026 all eight automated suites for the application and route engine passed (247 checks). On a local server, routes took a median of 93 ms and floor changes under 0.1 s, but a first visit over a simulated 4G connection with a slowed processor took 3.4 s, which misses the 2-second target. Sign-in uses demonstration accounts only, and no real phone or field test with students has been carried out. These are the main limitations and the next steps.

Contents

Declaration i

Acknowledgements ii

Abstract iii

1 Introduction 1

1.1 Background of the Study 1

1.2 Problem Statement 1

1.3 Research Aim and Objectives 2

1.4 Research Questions 3

1.5 Significance of the Study 3

1.6 Scope and Limitations 3

1.7 Structure of the Report 4

2 Literature Review 5

2.1 Critical Review of Key Related Studies 5

2.1.1 Campus navigation applications 5

2.1.2 Commercial and institutional campus maps 6

2.1.3 Route finding 7

2.1.4 Timetables, web applications and supporting standards 7

2.1.5 Comparison and research gap 8

3 Research Methodology **9**

3.1 Research Design 9

3.1.1 System design 9

3.1.2 Timeline and team contributions 12

3.1.3 Data collection methods and description of data sources 13

3.1.4 Rationale for using secondary data 13

3.1.5 Data selection criteria 13

3.1.6 Data analysis procedures 14

3.1.7 Evaluation of data quality 14

3.2 Requirements Catalogue and Feature Selection 15

3.2.1 Descoped requirements 16

3.3 Data Analysis Techniques 17

3.4 Ethical Considerations 18

4 Results and Analysis **19**

4.1 Presentation of Key Findings 19

4.1.1 The application 19

4.1.2 Change C1: BAC Maps joined to the finished map and Pathfinder 22

4.1.3 The route-finding algorithm 23

4.1.4 Change C2: the timetable on the profile 24

4.1.5 Change C3: routes kept clear of the panel 25

4.1.6 Change C4: the interface rebuilt in React 26

4.1.7 Change C5: defects found in testing and fixed 26

4.1.8 Test results 26

4.1.9 Offline behaviour 31

4.2 Data Interpretation 31

4.3 Linking Findings to the Research Questions 32

5 Discussion **34**

5.1 Critical Discussion 34

5.2 Comparison with Previous Studies 35

5.3 Explanation of Unexpected Results 35

6 Conclusion and Recommendations **37**

6.1 Summary of Key Findings 37

6.2 Contributions to Knowledge 38

6.3 Practical Implications 38

6.4 Limitations 38

6.5 Recommendations for Future Research 39

References **40**

A Abbreviations **43**

B Requirement Cards **44**

C Test Commands and Run Instructions **46**

List of Figures

3.1 Architecture of BAC Maps, including Pathfinder, both adapters, the room data and the drawings 10
3.2 Use case diagram. Dashed use cases are planned, partial or descoped . . 10
3.3 Activity diagram for getting directions 11
3.4 Entity-relationship diagram of the demonstration database 12

4.1 Overview of the main screens of BAC Maps 19
4.2 Finding a place: search, directory, place card and floors 20
4.3 Directions: steps, alternative route, changed start and the no-route message 20
4.4 The profile, timetable and appearance settings 21
4.5 The application at phone size (390×844, emulated in Microsoft Edge) . . 21
4.6 C1, the map before and after joining the finished map 22
4.7 C1, Directions before and after joining Pathfinder 22
4.8 C2, the profile before and after adding the class-group timetable 24
4.9 C3, the route before and after placing it in the free part of the map . . . 25
4.10 C4, structure of the interface code before and after the React rebuild . . 27

List of Tables

2.1 Comparison of reviewed systems with BAC Maps 8

3.1 Timeline and team contributions 12

3.2 Revised functional requirements 15

3.3 Requirements from the original catalogue that were descoped or not implemented 16

3.4 Non-functional requirements (revised) 17

4.1 Defects found in testing and fixed 28

4.2 Automated test suites, 24 October 2026 28

4.3 Test cases and results 29

4.4 Response times in milliseconds (median of five runs, range in brackets) . 30

4.5 Security checks against the demonstration server 30

4.6 Status of the revised requirements 31

B.1 Acceptance criteria for the revised functional requirements 44

Listings

4.1 Correcting the drawings' paths for the website (frontend/src/lib/map-adapter.js, lines 64–71) 23

4.2 Finding the gates from the verified walks (frontend/src/lib/nav-adapter.js, lines 51–66) 23

4.3 A* estimate and edge relaxation in Pathfinder (web/nav-engine.js, lines 694–703) 24

4.4 Upgrading an older database (server/db.py, lines 83–89) 24

4.5 Free areas of the map around the panel (frontend/src/components/App.jsx, lines 73–89) 25

4.6 Sign-in on the demonstration server (server/server.py, lines 165–175) . . 31

Chapter 1

Introduction

1.1 Background of the Study

Botswana Accountancy College's campus is a group of single-storey blocks, three hostels, parking areas and a two-storey Main Building that holds most laboratories, lecture rooms and offices. Rooms are named on a published timetable (for example *MC-Lab 1/017*), but nothing on campus links such a code to a place or a path. A student new to the college therefore has to ask, and a room on the first floor of the Main Building can only be reached through particular doors and staircases.

The project started as BSBS Maps, a group project to give BAC an interactive campus map. The first version of the website (named BAC Maps from here on) showed an approximate drawing of the campus and promised routes in a later release. In parallel, a separate part of the project produced an accurate campus map built from satellite traces and measured walks, together with a route engine called Pathfinder. This report documents the version in which these parts were joined and completed.

1.2 Problem Statement

Students and visitors cannot find a room from its timetable code without help, because:

1. no map shows the rooms inside the Main Building or which floor they are on;
2. no tool gives walking directions on campus that respect walls, doors and stairs;
3. timetable information is not linked to places on a map.

Before this version, BAC Maps showed only an approximate map, gave no routes (Figure 4.7) and did not show a student's class-group timetable.

1.3 Research Aim and Objectives

The aim is to design, build and evaluate a web application that helps students and visitors find places on the BAC campus, including rooms on both floors of the Main Building, and walk to them. The objectives are measurable so that Chapter 6 can give a verdict on each:

O1 Show the whole campus and both floors of the Main Building on one interactive map, with every mapped room searchable (FR-01 to FR-06).

O2 Compute walking routes from the Main Gate, a chosen place or the user's position to any reachable room, crossing no wall, and confirm this with automated wall checks every 25 cm (FR-07 to FR-14).

O3 Show a signed-in student's class-group timetable and own classes, linked to rooms on the map (FR-15 to FR-19).

O4 Respond within 2 seconds for map display, floor change and route display (NFR-01), and keep every button at least 44 px tall on phone and desktop (NFR-07).

O5 Keep account data private to its owner and keep the demonstration server's own files unreachable (NFR-04).

1.4 Research Questions

RQ1 Can a browser-based application compute walking routes across two floors that never pass through a wall?

RQ2 Can a student's published class-group timetable be linked to mapped rooms so that the next class can be found?

RQ3 Does the interface remain usable at phone and desktop sizes, with the route visible beside the directions panel?

RQ4 Does the application meet the 2-second response target on a local server and on a slower simulated connection?

1.5 Significance of the Study

For students, the application replaces asking for directions with a route on a phone. For the college, the method shows that a usable indoor and outdoor campus map can be made from free satellite imagery, a few measured walks and existing drawings, without surveying equipment or a paid mapping platform. The route engine and data pipeline can be reused when rooms or buildings change, because positions are kept in data files rather than in code.

1.6 Scope and Limitations

In scope: the BAC campus as traced in Google Earth; the Main Building's two floors and 40 rooms; walking routes; the five Computer Systems Engineering Year 3 class groups

in the 2026–2027 Semester 1 timetable export; a demonstration sign-in with dummy accounts.

Out of scope or limited:

- Sign-in is a demonstration with dummy accounts. There is no sign-up and no link to the college's own accounts.
- Interiors exist only for the Main Building. Other buildings are reached at their entrances.
- Room positions are approximate (Section 3.1.7). Office 3 has no door on the map, so no route reaches it.
- The camera-based door recognition in the original requirements was descoped (Section 3.2.1).
- Testing used a desktop browser, including at phone size. No real phone or field test with students was carried out.

1.7 Structure of the Report

Chapter 2 reviews related work. Chapter 3 describes the research design, the data sources, the requirements catalogue and the evaluation methods. Chapter 4 presents the application, the five changes made in this version and the test results. Chapter 5 discusses the findings, and Chapter 6 concludes with recommendations. The appendices contain abbreviations, the full requirement cards, test commands and run instructions.

Chapter 2

Literature Review

2.1 Critical Review of Key Related Studies

2.1.1 Campus navigation applications

Helmi et al. (2022) describe *Find 4 Me*, an indoor navigation application for Management and Science University in Malaysia aimed at new students and visitors. Beyond directions, it offers 360-degree views, venue booking, live chat and feedback, and was developed with the Spiral model. The study's evaluation reports that the outcome matched expectations, but the abstract gives no measured accuracy or user numbers, so its effectiveness cannot be compared directly. Its starting problem, new students unable to find classrooms in a large campus, is the same as BAC's.

Patel et al. (2025) present a campus application that combines outdoor navigation through a Mapbox plugin with indoor navigation built in Unity, with categories such as food courts and Firebase for data and sign-in. The design splits the campus into two systems, one per environment. BAC Maps instead uses one map and one route engine for indoor and outdoor walking, so a route from the gate to a first-floor laboratory is a single path rather than a hand-over between two tools.

Ausmeier et al. (2012), presented in Gaborone, compare two ways of capturing building data for phone-based indoor navigation: annotating floor plans on a desktop, and mapping a building by counting steps with the phone's accelerometer. Plan annotation was usable and scaled well, while step-based mapping gave distances too inaccurate for building a map, although small errors were acceptable when directing users. This supports BAC Maps' choice to build the map from traced plans and drawings, and to use measured walks only to confirm paths, not to derive geometry.

2.1.2 Commercial and institutional campus maps

Mappedin describes itself as a platform for indoor mapping, with software development kits for web, React Native, Android and iOS, and examples that include directions and search (Mappedin, 2026). Mapbox GL JS is a JavaScript library for interactive maps in the browser (Mapbox, 2026). Both provide mature rendering and routing, but depend on a paid or account-based service and on map data prepared in their formats. The University of British Columbia's wayfinding map offers directions between two places with walking, accessible and cycling modes, and can zoom to the user's location (University of British Columbia, 2026). It shows what an institution with a full geographic database can offer. BAC has no such database, which is the gap this project fills with traced data.

Illustrated isometric campus maps, such as those by Rod Hunt, are attractive and easy to read (Hunt, 2026), but a static picture cannot answer *how do I get there*. BAC Maps keeps an illustrated look by using the college's own campus drawings as the map, and places routes on them through a measured alignment (Section 3.1.7).

2.1.3 Route finding

A* search finds a minimum-cost path in a graph by expanding nodes in order of $f(n) = g(n) + h(n)$, where $g(n)$ is the cost so far and $h(n)$ an estimate of the remaining cost. It is guaranteed to find the cheapest path if h never overestimates (Hart et al., 1968). Pathfinder uses A* with a straight-line estimate scaled by the cheapest surface cost, which keeps the estimate admissible (Section 4.1.3).

2.1.4 Timetables, web applications and supporting standards

Products such as MyTimetable offer personalised timetables integrated with an institution's scheduling system (MyTimetable, 2026). BAC Maps uses a simpler approach suited to a prototype: the published timetable export is converted at build time into a file of class groups, each linked to mapped rooms.

The application is delivered as a website that can be installed and used offline through a service worker, a W3C specification that lets a page cache its files and serve them without a network (W3C, 2026b). A user's position comes from the W3C Geolocation API (W3C, 2026a). Accessibility targets follow the Web Content Accessibility Guidelines 2.2, which include a minimum target size for controls (W3C, 2024). The demonstration server stores passwords with PBKDF2, as specified in PKCS #5 (Moriarty et al., 2017), and exposes a small JSON API between the interface and the database, a separation also described by Guntupally et al. (2018) for a REST service in front of a database.

2.1.5 Comparison and research gap

Table 2.1 compares the reviewed systems. None of them gives a free, browser-based route across indoor floors and outdoor paths for a campus without an existing geographic database, while also linking each student's class-group timetable to rooms.

Table 2.1: Comparison of reviewed systems with BAC Maps

System	Indoor	Outdoor	One route both	Timetable	Free, browser
Find 4 Me (Helmi et al., 2022)	Yes	Not stated	Not stated	No	No (app)
Patel et al. (2025)	Yes (Unity)	Yes (Mapbox)	No	No	No (app)
Ausmeier et al. (2012)	Yes	No	No	No	No (app)
Mappedin, Mapbox	Yes / partly	Yes	Platform-dependent	No	No (licence)
UBC wayfinding	Not stated	Yes	Not stated	No	Yes
BAC Maps	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Chapter 3

Research Methodology

3.1 Research Design

The project followed an iterative prototype design: each version was built, tested with automated suites and in the browser, and corrected before the next. The version reported here is the fourth: the accurate map and Pathfinder (built first), the first BAC Maps website, the merge of the two, and the React rebuild. The code is in the group's Git repository, on branch `react-frontend` at commit `bb8fe1b` (the merge is commit `f363c57` on `main`).

3.1.1 System design

Figure 3.1 shows the architecture. The interface talks to the map only through `map-adapter.js` and to Pathfinder only through `nav-adapter.js`, so either can change without touching the interface. Both read the same room data, and the map draws the two campus drawings. A small Python server stores accounts, classes and saved places in SQLite, and reads the class groups from `timetables.json`. Figure 3.2 shows the use cases, with planned, partial and descoped items drawn dashed; Figure 3.3 shows the flow for getting directions; Figure 3.4 shows the database.

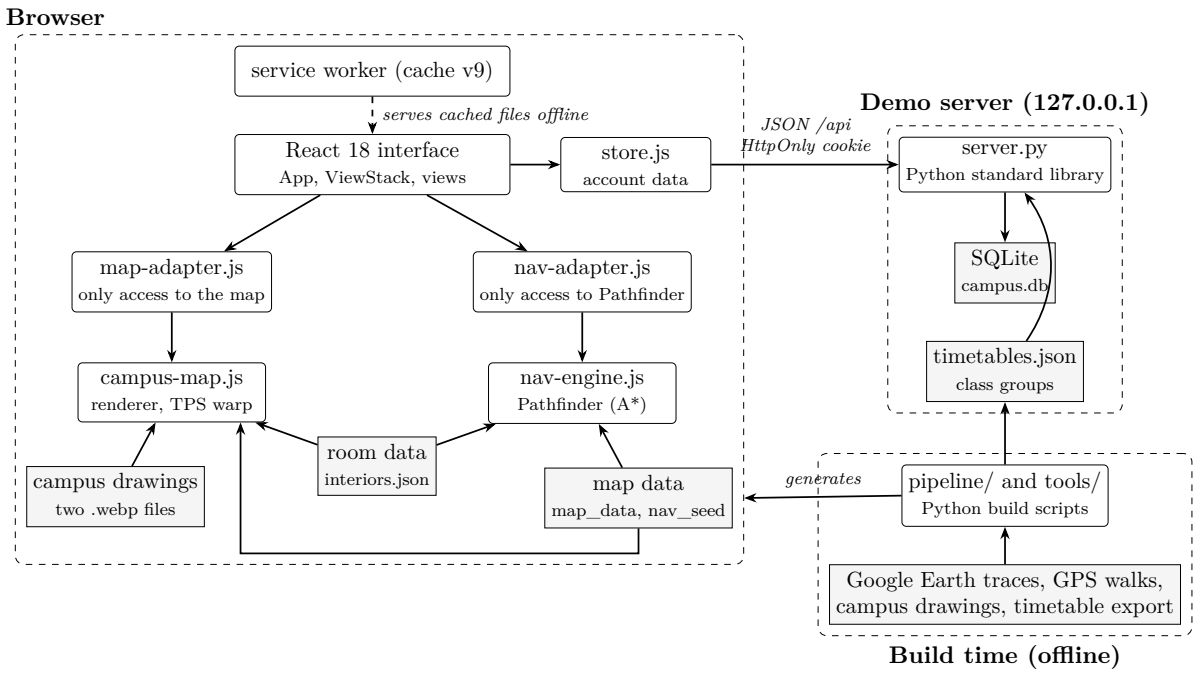


Figure 3.1: Architecture of BAC Maps, including Pathfinder, both adapters, the room data and the drawings

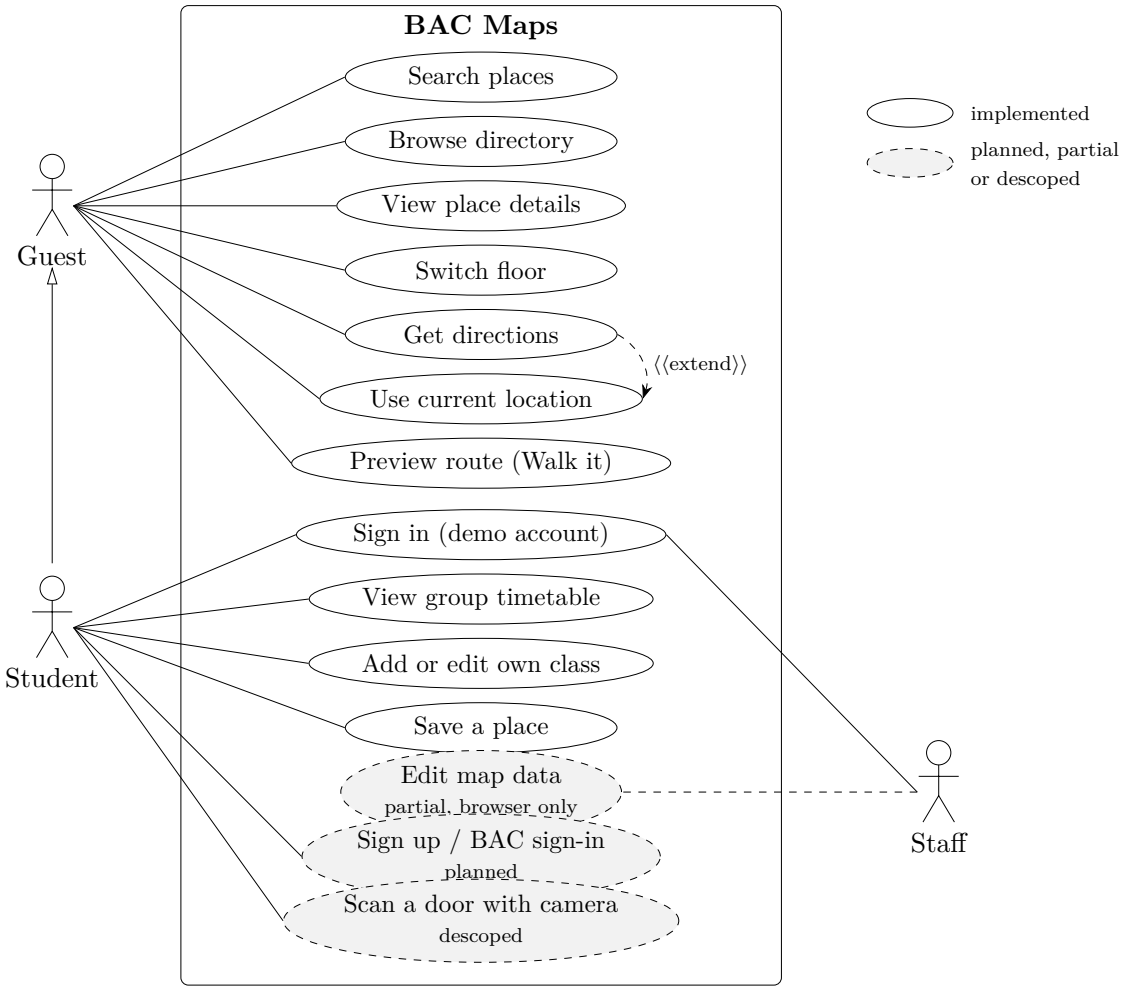


Figure 3.2: Use case diagram. Dashed use cases are planned, partial or descoped

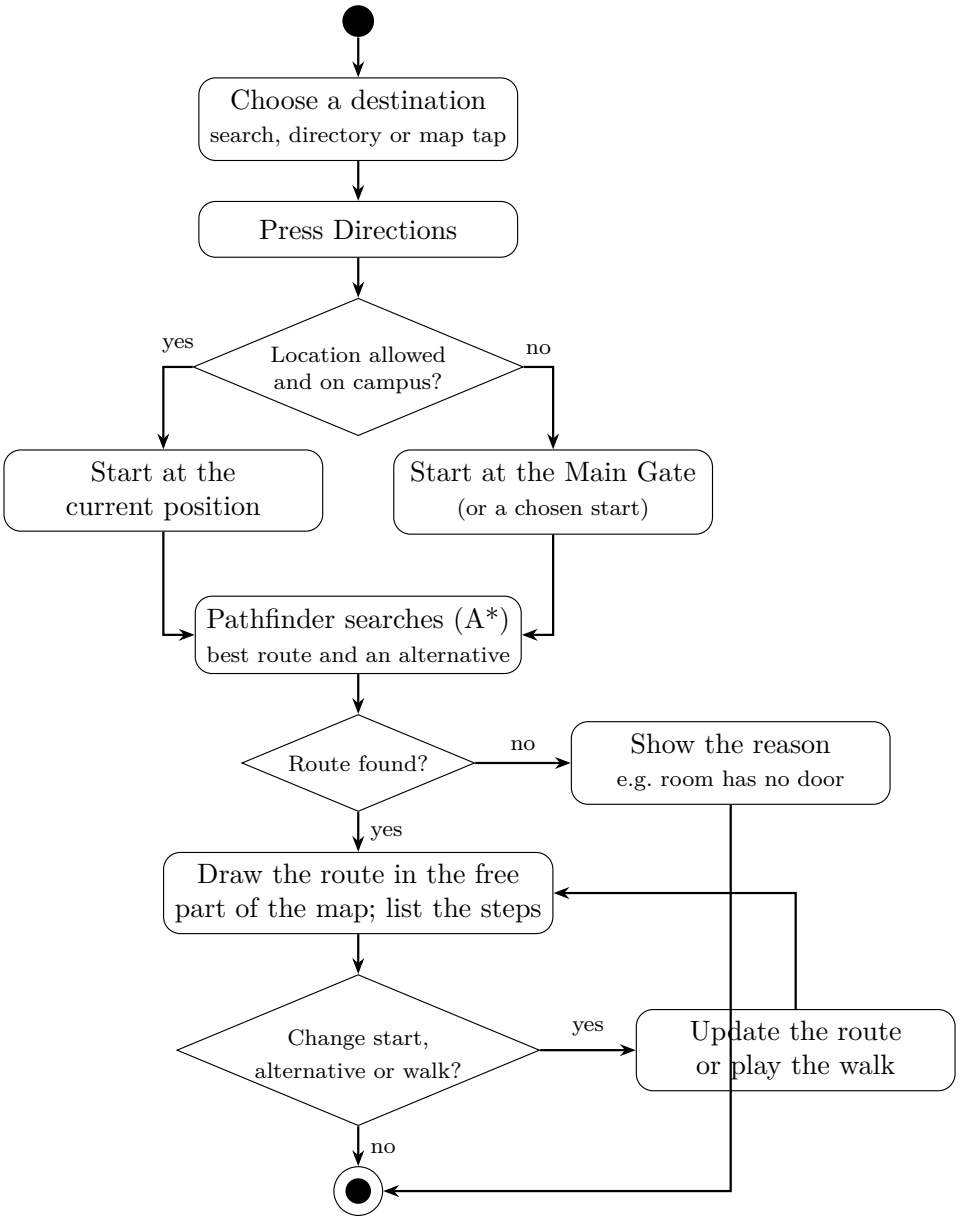


Figure 3.3: Activity diagram for getting directions

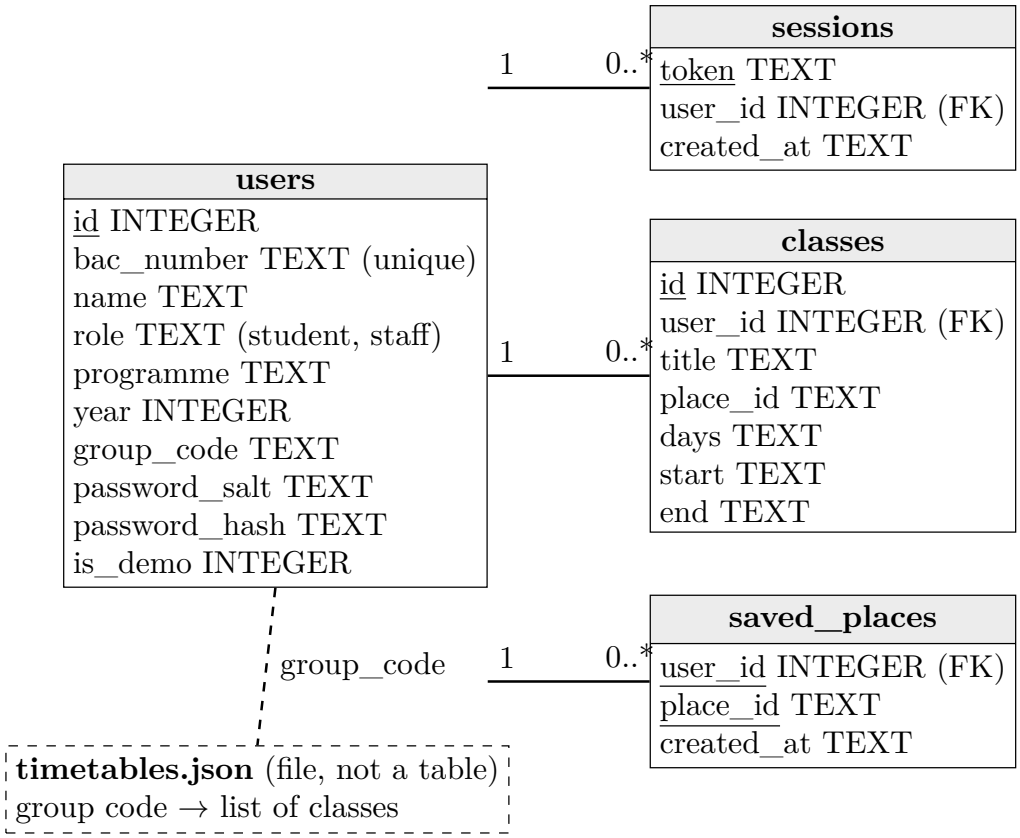


Figure 3.4: Entity-relationship diagram of the demonstration database

3.1.2 Timeline and team contributions

[INSERT project timeline: the main phases and their dates (Table 3.1)]

Table 3.1: Timeline and team contributions

Team member	Main contributions
Kaelo Kealeboga Selome	**[INSERT contributions]**
Monyana Bianca Kedukaetswe	**[INSERT contributions]**
Larona Mothame	**[INSERT contributions]**
Katlo Tutwane	**[INSERT contributions]**
Abigail Mototegi	**[INSERT contributions]**

3.1.3 Data collection methods and description of data sources

- **Google Earth traces:** 22 buildings, 18 paths, 10 areas and 3 roads traced over satellite imagery and exported as KML.
- **Measured walks:** ten walks recorded on site with GPS (for example *Main Gate to the student entrance*), marked as verified paths.
- **Campus drawings:** an illustrated campus drawing and a ground-floor drawing of the Main Building, both marked as concepts not to scale, used for what is where (room names, order, stairs).
- **Timetable export:** the SCIS 2026–2027 Semester 1 timetable exported from primetimetable.com, giving five class groups and 40 weekly classes.

3.1.4 Rationale for using secondary data

Surveying the campus or modelling the buildings was not possible within the project's time and budget. Satellite imagery, the college's drawings and its published timetable already existed, cover the whole campus and can be updated when the college changes. The measured walks were collected to check the secondary data where it matters most, on the main approaches from the gate.

3.1.5 Data selection criteria

A feature was included when it could be seen on the imagery or a drawing and named by the team. Room names were taken from the drawings; a room's shape was taken from the traced building footprint, because the drawings are not to scale. A staircase was connected only when both drawings agree: the stairs between the CAF and the lecture

theatre appear on only one drawing and are therefore left out until confirmed. Only timetable rooms that match a mapped place are linked (five room codes).

3.1.6 Data analysis procedures

Python build scripts turn the KML traces and walks into `map_data.json`, build the rooms, doors and stairs into `interiors.json`, and convert the timetable export into `timetables.json`. Rooms are made by dividing the Main Building's footprint, minus a 2.8 m corridor (measured on site), between the rooms named on the drawings. The drawings are placed on the map with a thin-plate spline (Bookstein, 1989) fitted to 82 control points:

$$\mathbf{p}(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b} + \sum_{i=1}^{n} \mathbf{w}_i \, r_i^2 \ln r_i, \qquad r_i = \|\mathbf{x} - \mathbf{c}_i\| \tag{3.1}$$

where $\mathbf{x}$ is a ground position in metres, $\mathbf{p}(\mathbf{x})$ the matching pixel on the drawing, $\mathbf{A}$ and $\mathbf{b}$ an affine transform, $\mathbf{c}_i$ the $n = 82$ control points and $\mathbf{w}_i$ their fitted weights. Equation 3.1 only places the existing drawings and routes onto each other; it does not draw the map.

3.1.7 Evaluation of data quality

The fitted drawing lies a median of 1.6 m and at most 13.2 m from its control points. The campus drawing matches 17 traced building labels with a median error of 1.5 m (worst 6.7 m). All 40 rooms are marked approximate: the drawings place the south-east wing 4–8 m east of the satellite footprint, and rooms there were fitted to the footprint. Storey counts for buildings other than the Main Building are assumed to be one, and two stair positions are estimated from an evacuation sign. Office 3 has no corridor wall in the footprint, so it has no door.

3.2 Requirements Catalogue and Feature Selection

The original catalogue (FR-01 to FR-13, NFR-01 to NFR-09) was built around a camera that would recognise a room's door and check it against the timetable. That system was not implemented. The catalogue was therefore revised to the 20 functional requirements in Table 3.2, prioritised with MoSCoW (Must, Should, Could, Won't). Each revised requirement keeps a link to the original identifier it replaces. The descoped requirements are listed in Table 3.3, and the non-functional requirements in Table 3.4. Appendix B gives the full cards with acceptance criteria.

Table 3.2: Revised functional requirements

ID	The system shall …	Priority	Original
FR-01	display an interactive map of the campus that can be panned, zoomed and reset	Must	FR-01
FR-02	switch between All, Ground and First floor without reloading	Must	FR-02
FR-03	search buildings, rooms and facilities by name	Must	new
FR-04	offer quick category picks and a directory grouped by category	Should	new
FR-05	select a building or room by tapping it on the map	Should	new
FR-06	show a place card with category, building and floor, and actions for directions, saving and showing on the map	Should	FR-06 (revised)
FR-07	compute a walking route from an entrance gate to a chosen room	Must	FR-05
FR-08	route across floors through mapped doors and stairs, with steps grouped by floor	Must	FR-05
FR-09	offer an alternative route when a distinct one exists	Could	new
FR-10	let the user change the start and destination of a route	Should	new
FR-11	start from the user's position when on campus, otherwise from the Main Gate	Could	new
FR-12	preview a route with an animated walk and pin a selected step on the map	Could	new
FR-13	explain why no route exists when a place cannot be reached	Must	new
FR-14	keep the shown route clear of the directions panel	Should	new (C3)
FR-15	let a student sign in and out so data is tied to one account	Must	FR-12

ID	The system shall …	Priority	Original
FR-16	show a signed-in student's class-group timetable by day	Must	FR-03 (revised)
FR-17	let a student add, edit and delete their own classes	Should	FR-03
FR-18	show the student's next class with directions to its room	Must	FR-04 (revised)
FR-19	let a student save places and list them in the profile	Should	new
FR-20	let staff correct map data (doors, links) without changing code	Could	FR-11 (revised)

3.2.1 Descoped requirements

The camera requirements FR-07, FR-08 and FR-09 were rated Must in the original catalogue. They were descoped because the prerequisites do not exist in the project: there is no database of door photographs for the 40 rooms, no vision service has been chosen or tested, and sending camera images of the campus to an outside service would need the college's permission. Delivering the map and routes, on which the camera feature would also depend, was given priority. [**INSERT the team's own recorded reason and date for descoping the camera, if different**]

Table 3.3: Requirements from the original catalogue that were descoped or not implemented

Original	Requirement	Priority	Status and reason
FR-07	Capture a live frame of a door with the camera	Must	Descoped (see above)
FR-08	Match a door image against room photos with a vision API (≥90% correct)	Must	Descoped; no photo set
FR-09	Cross-check a recognised room against the timetable	Must	Descoped; depends on FR-08
NFR-02	Camera result within 3 seconds	–	Descoped with FR-07 to FR-09
FR-10	Arrival alert before the next class	Should	Not implemented
FR-13	Show room capacity when vacant	Could	Not implemented; no capacity data

Table 3.4: Non-functional requirements (revised)

ID	Requirement
NFR-01	The map shall display, a floor change shall take effect and a route shall appear within 2 seconds.
NFR-03	The application shall be usable on Android and iOS phones and on desktop browsers.
NFR-04	Account data shall be stored securely and be readable and changeable only by its owner.
NFR-05	The system shall remain available during peak class-changeover periods.
NFR-06	Rooms and paths shall be updatable through the build pipeline without code changes.
NFR-07	The interface shall follow accessible design practice: contrast, text size and controls at least 44 px tall.
NFR-08	The system shall use React 18 with Vite for the interface, JavaScript for the map and route engine, Python for the build scripts and demonstration server, and SQLite for account data.
NFR-09	The prototype shall be delivered within the 16-week project timeline.
NFR-10	After one visit, the map and routes shall work without a network connection (new).

3.3 Data Analysis Techniques

- **Automated suites:** five Node.js suites test Pathfinder (navigation, interiors, scenarios, walks, concurrency) and three Playwright suites test the application in Microsoft Edge (site, profile, floors). The site and profile suites start the demonstration server with a temporary database.

- **Wall sampling:** every route is sampled every 25 cm and each sample is checked to lie on walkable ground and outside rooms except through doors.

- **Visibility measurement:** the share of the route's on-screen length that is not covered by the panel, measured on desktop and phone sizes.

- **Timings:** measured inside the page with `performance.now()`, five runs per case, cache disabled, on a local server and with a simulated 4G-like connection (9 Mbit/s

down, 1.5 Mbit/s up, 60 ms latency) and a 4× processor slowdown.

- **Security checks:** direct API calls with wrong passwords, a second account, invalid input and requests for private files.
- **Manual checks:** features without an automated check (marked *manual* in Table 4.3) were exercised by hand in Edge and recorded in the screenshot tour.

3.4 Ethical Considerations

Only dummy demonstration accounts exist; no student's real data is stored. Passwords are stored as salted PBKDF2-SHA256 hashes with 200 000 rounds, and the session is an HttpOnly, SameSite=Strict cookie that expires after seven days. The user's position is requested through the browser's permission prompt, used only in the browser to choose the start of a route, and never sent to the server. The timetable lists lecturers by initials only. [**INSERT whether the college gave permission to use its campus drawings and timetable export**]

Chapter 4

Results and Analysis

4.1 Presentation of Key Findings

4.1.1 The application

Figure 4.1 gives an overview of the application's main screens. On desktop, a search bar and a panel sit over the map; on a phone, the panel becomes a card at the bottom of the screen. Figures 4.2 to 4.5 show the features in the revised catalogue.

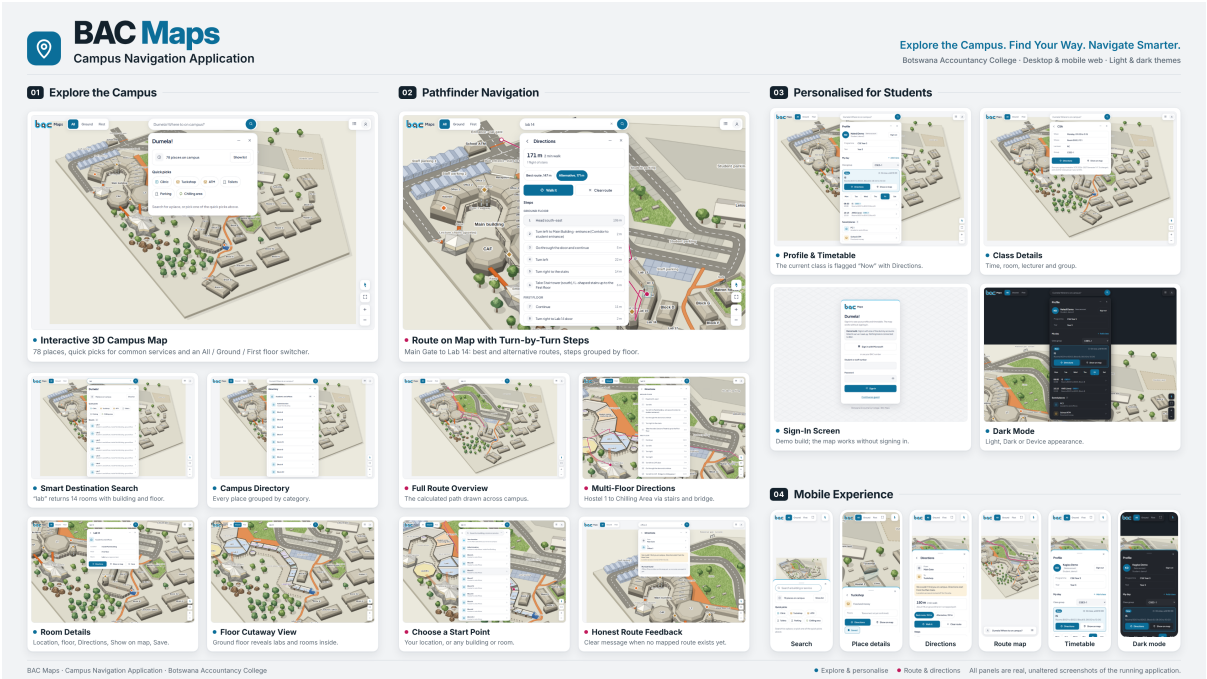
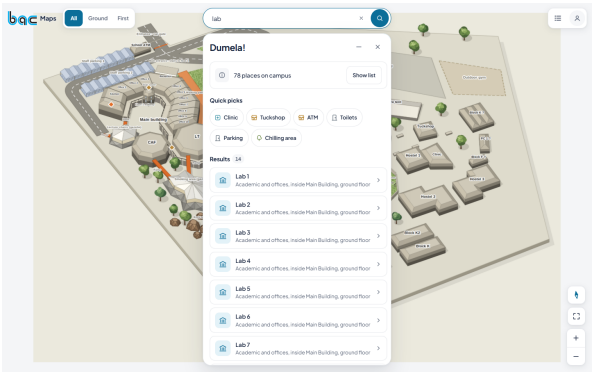
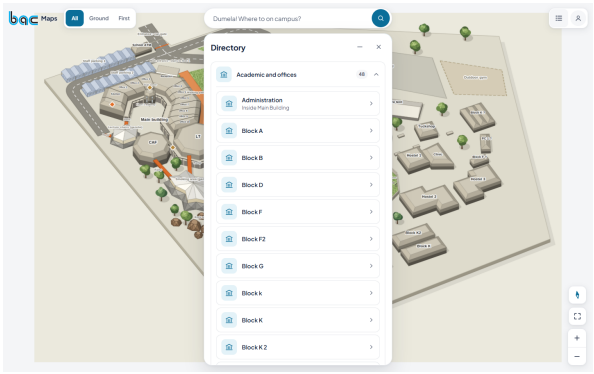


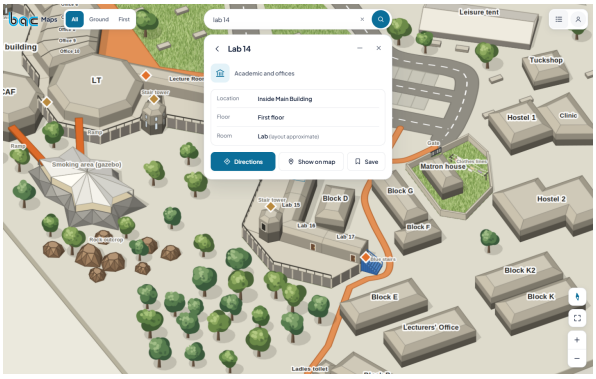
Figure 4.1: Overview of the main screens of BAC Maps



(a) Search results for “lab” (FR-03)



(b) Directory by category (FR-04)



(c) Room card for Lab 14 (FR-06)

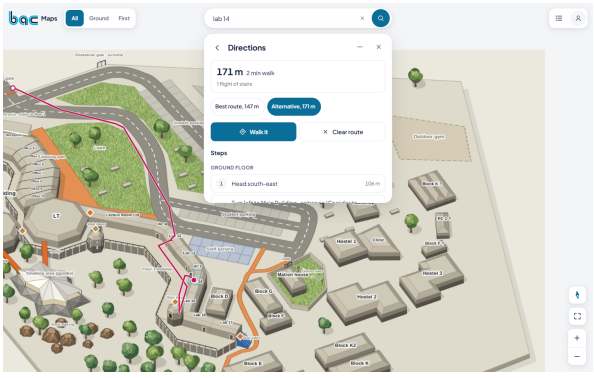


(d) Ground floor shown (FR-02)

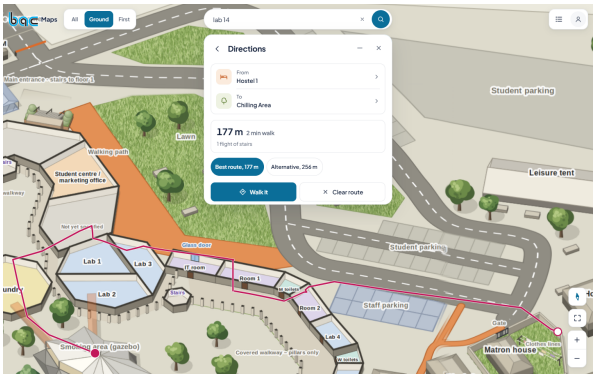
Figure 4.2: Finding a place: search, directory, place card and floors



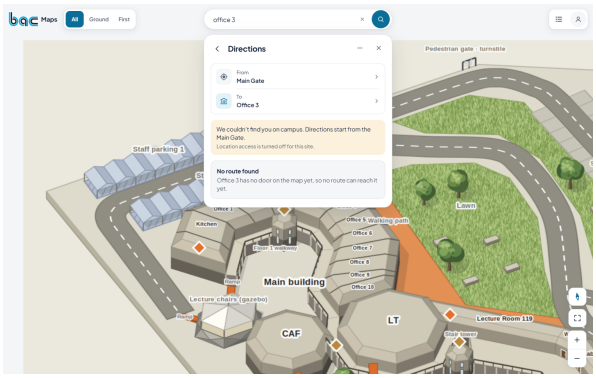
(a) Steps grouped by floor (FR-08)



(b) Alternative route (FR-09)

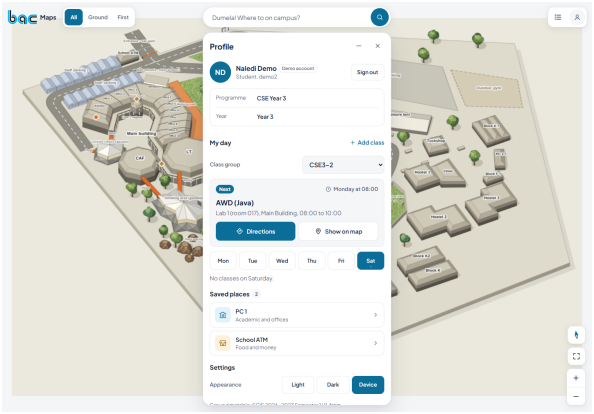


(c) Changed start and end (FR-10)

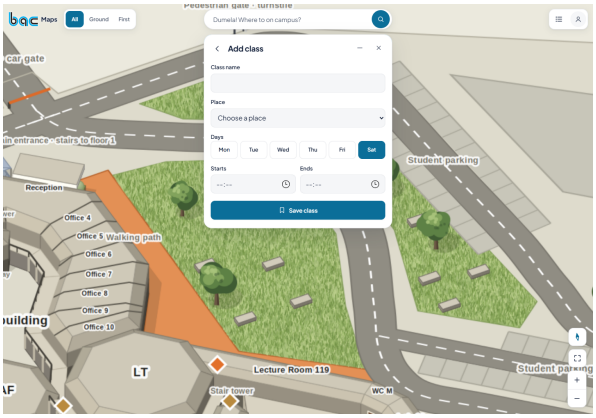


(d) No route to Office 3 (FR-13)

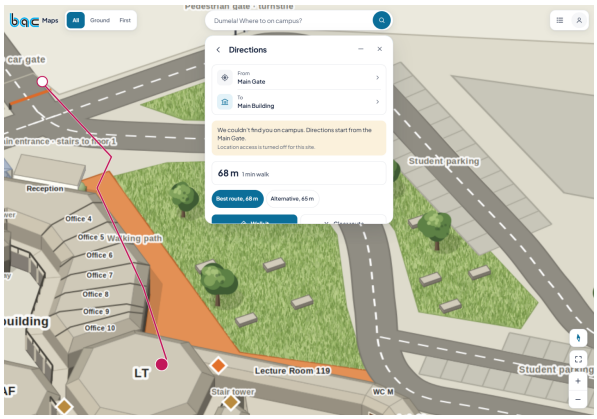
Figure 4.3: Directions: steps, alternative route, changed start and the no-route message



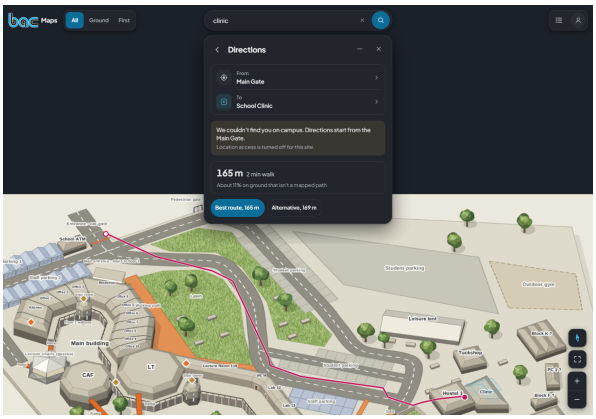
(a) Profile with group timetable (FR-16, FR-18)



(b) Adding an own class (FR-17)



(c) Directions to a class (FR-18)

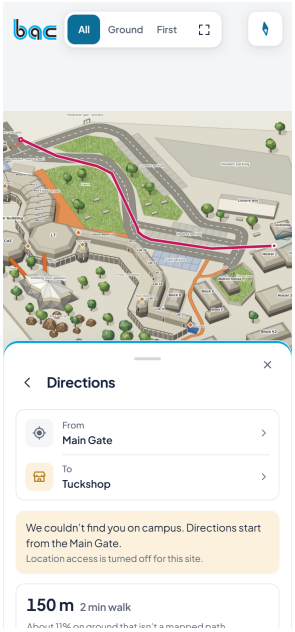


(d) Dark appearance (NFR-07)

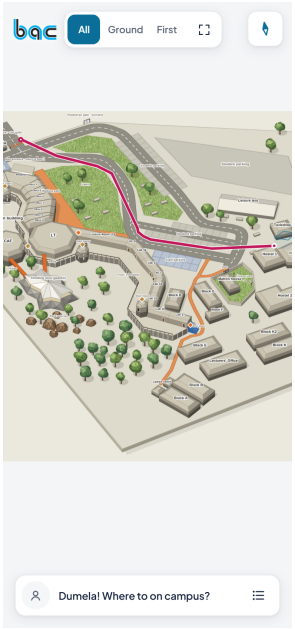
Figure 4.4: The profile, timetable and appearance settings



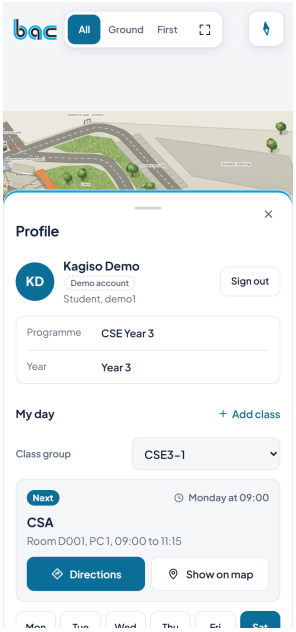
(a) Home



(b) Directions



(c) Route on map



(d) Timetable

Figure 4.5: The application at phone size (390×844, emulated in Microsoft Edge)

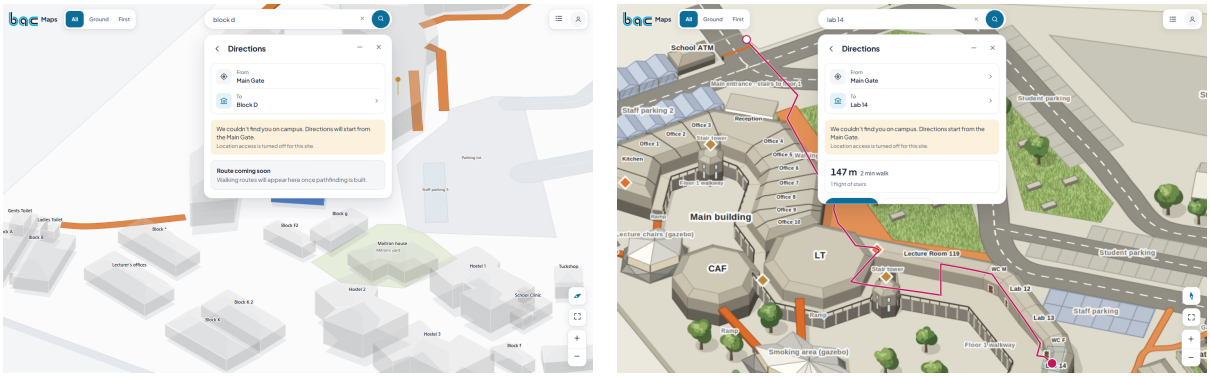
4.1.2 Change C1: BAC Maps joined to the finished map and Pathfinder

Before this change, BAC Maps showed an approximate block map and its Directions panel only said that routes were coming (Figure 4.6). The interface was connected to the campus drawings and to Pathfinder through the two adapters, so the map now shows the real campus, both floors and 40 rooms, and Directions returns real routes. Listing 4.1 corrects the drawings' file paths, which was the first defect found (Table 4.1, D1). Listing 4.2 finds the Main Gate and Student Entrance from the ends of the verified walks, because the traced data has no gate points.



(a) Before: approximate map (b) After: the campus drawings

Figure 4.6: C1, the map before and after joining the finished map



(a) Before: "Route coming soon" (b) After: a Pathfinder route to Lab 14

Figure 4.7: C1, Directions before and after joining Pathfinder

Listing 4.1: Correcting the drawings' paths for the website (frontend/src/lib/map-adapter.js, lines 64–71)

```
function withDrawingsFrom(interiors, dir) {
  var renders = Object.assign({}, interiors.renders, {
    images: interiors.renders.images.map(function (im) {
      return /^(\/|[a-z]+:)/i.test(im.src) ? im : Object.assign({}, im, { src: dir + im.src });
    })
  });
  return Object.assign({}, interiors, { renders: renders });
}
```

Listing 4.2: Finding the gates from the verified walks (frontend/src/lib/nav-adapter.js, lines 51–66)

```
/* map_data.json has no gate or entrance points, but the walks verified on
   site start or end there and are named for it ("Main Gate to ...",
   "... to the student entrance"). Read the positions from those ends. */
function findLandmarks(data) {
  var gate = [], student = [];
  (data.paths || []).forEach(function (p) {
    if (!p.verified || !p.points_m || p.points_m.length < 2) return;
    var first = p.points_m[0], last = p.points_m[p.points_m.length - 1];
    if (/^main gate to /i.test(p.name)) gate.push(first);
    if (/ to the main gate\b/i.test(p.name)) gate.push(last);
    if (/ to the student entrance$/i.test(p.name)) student.push(last);
  });
  landmarks = {};
  if (gate.length) landmarks["Main Gate"] = median(gate);
  if (student.length) landmarks["Student Entrance"] = median(student);
}
```

4.1.3 The route-finding algorithm

Pathfinder builds a 0.5 m grid of the campus outdoors and a graph of corridors, doors and stairs indoors. Walking on a traced path costs its length, on a verified walk 0.8 times its length, and on untraced open ground 2.5 times; a flight of stairs costs 10.4 and adds 12 seconds. The search expands nodes in order of

$$f(n) = g(n) + h(n), \qquad h(n) = 0.8 \cdot \min_{t \in T} \|\mathbf{x}_n - \mathbf{x}_t\| \tag{4.1}$$

where $g(n)$ is the cost of the best path found to node n, $\mathbf{x}_n$ its position, T the set of goal nodes and 0.8 the cost per metre of the cheapest surface. Because no surface is cheaper than 0.8, h never overestimates and the route found is the cheapest (Hart et al., 1968). Listing 4.3 shows the estimate and the edge relaxation. For the alternative route, nodes

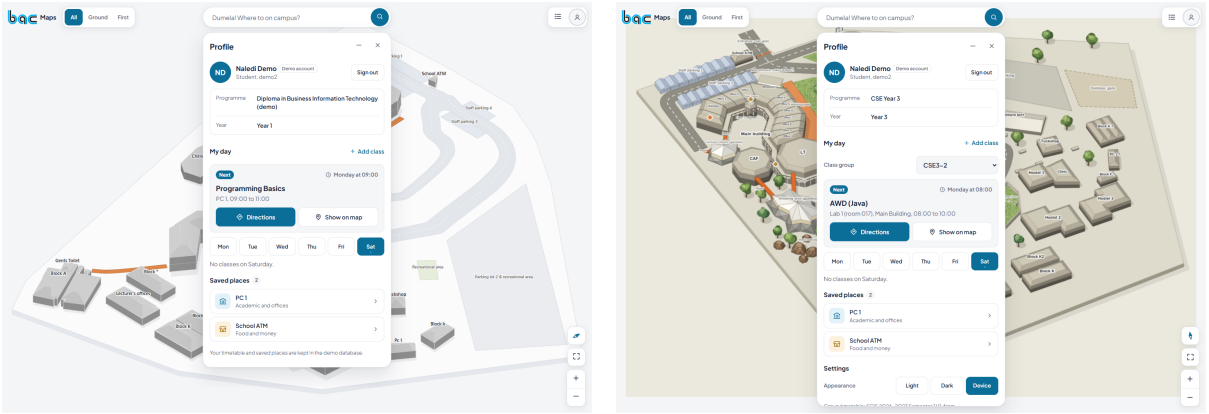
of the first route cost 2.2 times more, and a second search finds a route that differs from it. Searches run in 8 ms slices so that the page never freezes, and every route is checked against walls every 25 cm before it is shown.

Listing 4.3: A* estimate and edge relaxation in Pathfinder (web/nav-engine.js, lines 694–703)

```
    Search.prototype.h = function (u) {
      var x = px(u), y = py(u), m = Infinity;
      for (var i = 0; i < this.gl.length; i++) { var d = Math.hypot(this.gl[i][0] - x, this.gl[i][1] -
     y); if (d < m) m = d; }
      return m * VERIFIED_MULT;                              // the cheapest surface costs this per
     metre, so the estimate never overestimates
    };
    Search.prototype.relax = function (u, v, c) {
      var A = this.A, ng = A.G[u] + (this.pen && A.PEN[v] === this.pen ? c * PEN_MULT : c);
      if (A.STAMP[v] !== this.stamp) { A.STAMP[v] = this.stamp; A.G[v] = Infinity; }
      if (ng < A.G[v]) { A.G[v] = ng; A.PAR[v] = u; this.open.push(ng + this.h(v), v); }
    };
```

4.1.4 Change C2: the timetable on the profile

Before this change, the profile listed only classes the student had typed in, and students in older databases had no class group, so no timetable could be shown (Figure 4.8). The profile now shows the student's class-group timetable by day, with the next class and directions to it, and loading, error and empty states. Listing 4.4 gives existing demonstration students their group when an older database is upgraded.



(a) Before: own classes only

(b) After: group timetable

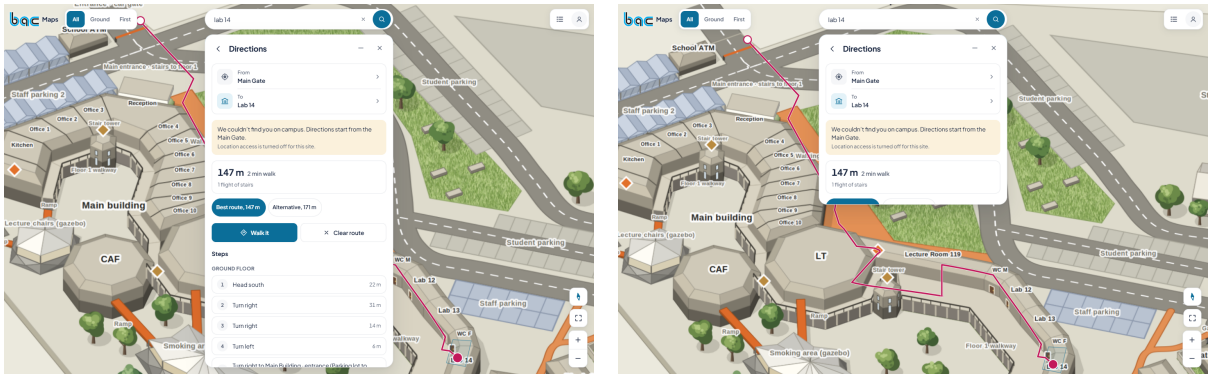
Figure 4.8: C2, the profile before and after adding the class-group timetable

Listing 4.4: Upgrading an older database (server/db.py, lines 83–89)

```
        # databases made before class groups existed
        import seed
        cols = [r["name"] for r in conn.execute("PRAGMA table_info(users)")]
        if "group_code" not in cols:
            conn.execute("ALTER TABLE users ADD COLUMN group_code TEXT")
            # their demo students would otherwise have no group, so no timetable
            seed.assign_groups(conn)
```

4.1.5 Change C3: routes kept clear of the panel

Routes were centred on the screen, so on desktop the panel covered much of them, and on a phone the card covered the lower half of the map. The route is now placed in the largest free part of the map beside or below the panel (Listing 4.5), and the panel becomes shorter while a route is shown. The before image in Figure 4.9 was reproduced on the current build by switching this placement off, because the earlier build cannot show routes at all. The site suite measures that 100% of the route is visible after the change on a phone.



(a) Before (reproduced): route under the panel (b) After: route beside the panel

Figure 4.9: C3, the route before and after placing it in the free part of the map

Listing 4.5: Free areas of the map around the panel (frontend/src/components/App.jsx, lines 73–89)

```
  function freeRects() {
    const m = mapRef.current.getBoundingClientRect();
    const rel = (r) => ({ left: r.left - m.left, top: r.top - m.top, right: r.right - m.left, bottom:
     r.bottom - m.top });
    const bar = rel(document.getElementById("topbar").getBoundingClientRect()), tools = rel(document.
     getElementById("map-tools").getBoundingClientRect());
    const wide = desktop.matches, open = latest.current.expanded;
    let top = bar.bottom, bottom = m.height, left = 0, right = m.width;
    if (wide) right = tools.left;                                  // zoom and compass run down the right
     edge
    else if (!open) bottom = rel(pillRef.current.getBoundingClientRect()).top;
```

```
    if (!open) return [{ left, top, right, bottom }];
    const c = rel(cardRef.current.getBoundingClientRect());
    if (!wide) return [{ left, top, right, bottom: c.top }];
    return [
      { left: left, top: top, right: c.left, bottom: bottom },          // left of the panel
      { left: c.right, top: top, right: right, bottom: bottom },        // right of it
      { left: left, top: c.bottom, right: right, bottom: bottom }       // below it
    ];
  }
```

4.1.6 Change C4: the interface rebuilt in React

The interface was rewritten from one 1 557-line script that built every view with DOM calls into React components built with Vite (Figure 4.10). The map, Pathfinder and their adapters keep the same logic, and the interface looks the same, so no before-and-after screenshot is given. The build writes its file names into the service worker, which now caches 26 files under version 9. Three defects appeared while porting and were fixed before release: an extra wrapper around icons broke their styling, the start picker's header and list did not share state, and the view stack hid views during their slide animation.

4.1.7 Change C5: defects found in testing and fixed

Table 4.1 lists the ten defects found while testing the merged application, with their causes and corrections.

4.1.8 Test results

All suites were run on 24 October 2026 on Windows 11 with Node.js 24.14.1, Python 3.12.10 and Microsoft Edge 155 (headless, through Playwright), against commit `bb8fe1b`. Table 4.2 gives the results. Phone results come from Edge at a phone size of 390×844 with touch emulation, not from a real phone.

Two older suites for the separate Pathfinder demonstration page (`e2e_core`,

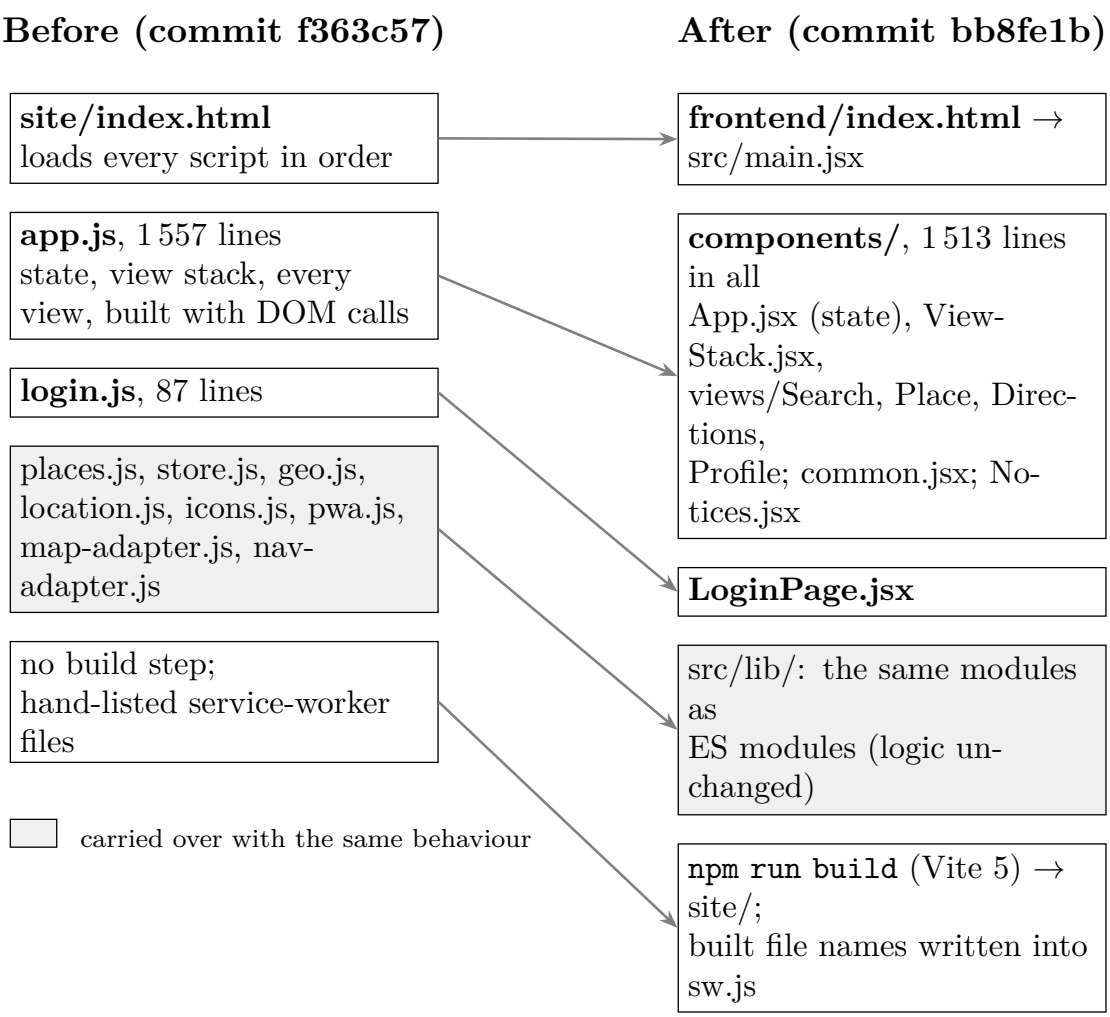


Figure 4.10: C4, structure of the interface code before and after the React rebuild

Table 4.1: Defects found in testing and fixed

No.	Observed problem	Cause	Correction
D1	The campus drawings did not load	Image paths pointed to the wrong folder	The map adapter corrects the paths
D2	The timetable was missing from the profile	The timetable work had never been merged in	Merged in, file by file
D3	Existing students still had no timetable	Upgrading the database left their group empty	Demo students receive their group on upgrade
D4	A signed-in student briefly appeared as a guest	The profile had no loading state	Loading, error and empty states added
D5	No routes in a background tab	Pathfinder waited for the screen to redraw	A timer now starts it as well
D6	The walking marker stayed after Clear route	Its clean-up was lost when the walk ended	Clean-up kept until the route is cleared
D7	The top search showed the previous results	The search view wrote the old text back	The new text is kept first
D8	Routes hidden behind the panel	Routes were centred on the screen	Placed in the free part of the map (C3)
D9	A route partly covered once the panel filled	Placed while the panel was still loading	Placed after the panel is drawn
D10	Outdated files appeared during testing	The offline cache kept serving them	Cache version raised; cache cleared between runs

Table 4.2: Automated test suites, 24 October 2026

Suite	What it checks	Passed	Failed
nav	grid, movement, routes outdoors	29	0
interiors	rooms, doors, stairs; no route through a wall	29	0
scenarios	demonstration routes, every position on walkable ground	24	0
walks	91 routes, none crossing a wall or leaving walkable ground	66	0
concurrency	overlapping and cancelled searches	2	0
e2e_site	search, directions, route visibility, button sizes, no script errors	35	0
e2e_profile	guest and signed-in profile, group timetable, loading and error states, next class	40	0
e2e_floors	Pathfinder page (same engine and data): floor controls; 1 482 routes checked against walls	22	0
Total		247	0

`e2e_perf`) do not complete: `e2e_core` passed 21 checks, failed 4 and then stopped. The same four failures occur on the commit before the merge (`fafb968`), so they come from that page's tests, not from this version's changes (Section 5.3).

Table 4.3 maps test cases to the requirements, and Table 4.4 gives the timings. Table 4.5 gives the security checks.

Table 4.3: Test cases and results

ID	Test	Req.	Evidence	Result
T-01	Map shows the campus drawings; zoom, pan and reset work	FR-01	e2e_site	Pass
T-02	All, Ground and First change the floor without reloading	FR-02	e2e_site, e2e_floors	Pass
T-03	Searching "first floor lab" lists the six first-floor labs	FR-03	e2e_site	Pass
T-04	Category chips and directory list the right places	FR-04	manual	Pass
T-05	Tapping a building opens its card	FR-05	e2e_site	Pass
T-06	Room card shows category, building and floor	FR-06	e2e_site	Pass
T-07	Route from the Main Gate to Lab 14 is found and drawn	FR-07	e2e_site	Pass
T-08	Route uses stairs; steps grouped by floor	FR-08	e2e_site, scenarios	Pass
T-09	No route sample crosses a wall (1 482 routes; 91 walk routes)	FR-08	e2e_floors, walks	Pass
T-10	Alternative route offered and selectable	FR-09	e2e_site	Pass
T-11	Changing start and destination recalculates	FR-10	e2e_site	Pass
T-12	With location blocked, the route starts at the Main Gate	FR-11	e2e_site	Pass
T-13	Position on campus used as the start (real phone)	FR-11	–	Not tested
T-14	Walk it animates the marker; Clear route removes it	FR-12	e2e_site	Pass
T-15	Office 3 gives a reason instead of a route	FR-13	e2e_site	Pass
T-16	100% of the route visible beside the panel on a phone	FR-14	e2e_site	Pass
T-17	Sign-in, sign-out; wrong password refused	FR-15	e2e_profile, Table 4.5	Pass
T-18	Group timetable shown by day, with loading and error states	FR-16	e2e_profile	Pass

ID	Test	Req.	Evidence	Result
T-19	Add, edit and delete an own class; invalid times refused	FR-17	manual, Table 4.5	Pass
T-20	Next class card with directions	FR-18	e2e_profile	Pass
T-21	Save and unsave a place	FR-19	manual	Pass
T-22	Staff edits saved and used by routes (Pathfinder page)	FR-20	manual	Partly
T-23	Response times within 2 s	NFR-01	Table 4.4	Partly
T-24	Another account cannot read or change a class	NFR-04	Table 4.5	Pass
T-25	Every visible button at least 44 px tall	NFR-07	e2e_site, e2e_profile	Pass
T-26	Map and route work offline after one visit	NFR-10	Section 4.1.9	Pass
T-27	Installation on a phone	NFR-10	–	Not tested

Table 4.4: Response times in milliseconds (median of five runs, range in brackets)

Measurement	Local server	4G-like network, 4× slower processor
Map ready (first visit)	1 394 (1 152–2 072)	3 439 (3 412–4 689)
Switch to ground floor	16 (10–22)	38 (31–48)
Switch to first floor	42 (24–95)	65 (49–93)
Route to Lab 14 shown	93 (86–129)	484 (450–546)
Data transferred	2 468 kB	2 495 kB

Table 4.5: Security checks against the demonstration server

Check	Response	Result
Sign-in with a wrong password, or an unknown number	401	Pass
Reading classes without a session, or after signing out	401	Pass
Second account lists classes: first account's class absent	not listed	Pass
Second account edits the first account's class	404	Pass
Second account deletes it: still present for its owner	still present	Pass
Invalid group or a class that ends before it starts	400	Pass
Session cookie flags	HttpOnly, SameSite=Strict	Pass
Database, seed file, Git config, raw timetable, build scripts, a path climbing out of `/site/`	404	Pass

4.1.9 Offline behaviour

After one visit with the service worker enabled, the browser held two caches with 26 files. With the network then switched off, the page reloaded, the map appeared and a route to Lab 14 was computed. Requests to the server failed as expected, so sign-in and saving need a connection. Installation on a phone was not tested, because it needs the site hosted over HTTPS.

Listing 4.6 shows how the server checks a password and starts a session.

Listing 4.6: Sign-in on the demonstration server (server/server.py, lines 165–175)

```
        if (method, name) == ("POST", "login"):
            body = self.read_json() or {}
            username = str(body.get("username", "")).strip()
            password = str(body.get("password", ""))
            row = conn.execute("SELECT * FROM users WHERE bac_number = ?", (username,)).fetchone()
            if not row or not db.check_password(password, row["password_salt"], row["password_hash"]):
                return self.send_json(401, {"error": "That number and password don't match a demo account."})
            token = secrets.token_urlsafe(32)
            conn.execute("INSERT INTO sessions (token, user_id) VALUES (?, ?)", (token, row["id"]))
            cookie = "%s=%s; Path=/; HttpOnly; SameSite=Strict; Max-Age=%d" % (COOKIE, token, SESSION_DAYS * 86400)
            return self.send_json(200, {"user": db.user_json(row)}, [("Set-Cookie", cookie)])
```

4.2 Data Interpretation

Table 4.6 gives the status of each requirement, with its evidence.

Table 4.6: Status of the revised requirements

ID	Status	Evidence and notes
FR-01	Met	T-01; Figure 4.6
FR-02	Met	T-02; Figure 4.2
FR-03	Met	T-03
FR-04	Met	T-04
FR-05	Met	T-05
FR-06	Met	T-06. Room photos and module titles from the original FR-06 are not shown; the lecturer's initials appear in class details only.
FR-07	Met	T-07; Figure 4.7
FR-08	Met	T-08, T-09

ID	Status	Evidence and notes
FR-09	Met	T-10
FR-10	Met	T-11
FR-11	Partly	T-12 passes; use of a real position on campus (T-13) not tested. GPS cannot tell floors apart, so a found position is taken as the ground floor.
FR-12	Met	T-14
FR-13	Met	T-15
FR-14	Met	T-16; Figure 4.9
FR-15	Partly	T-17 passes, but only with demonstration accounts; no sign-up or college sign-in.
FR-16	Met	T-18, for the five class groups in the export.
FR-17	Met	T-19
FR-18	Partly	T-20: the next class is shown in the profile with Directions; it is not highlighted on the map automatically, as the original FR-04 required.
FR-19	Met	T-21
FR-20	Partly	T-22: the Pathfinder page's editor saves corrections only in that browser and can export them; there is no shared staff database.
NFR-01	Partly	Floor changes and routes are well within 2 s. The first map display takes 1.4 s locally (one run 2.07 s) and 3.4 s on the slower profile.
NFR-03	Partly	Tested at phone size in Edge only; no Android or iOS device.
NFR-04	Met	T-24 and Table 4.5, for the demonstration server.
NFR-05	Not tested	No load test; the server runs locally for demonstration.
NFR-06	Met	Names, storeys and stairs are edited in the pipeline's configuration file; not rebuilt for this report.
NFR-07	Partly	T-25 passes; contrast was not measured with a tool.
NFR-08	Met	React 18.3, Vite 5.4, Python 3.12, SQLite.
NFR-09	[**INSERT status**]	[**INSERT whether the 16-week timeline was met**]
NFR-10	Met	T-26 on a local server; installation (T-27) not tested.

In total, 16 of the 20 functional requirements were met and four (FR-11, FR-15, FR-18, FR-20) partly met. Of the nine non-functional requirements with a status, four were met, four partly met and one not tested.

4.3 Linking Findings to the Research Questions

RQ1 is answered yes for the mapped data: no route in 1 482 checked routes, or in the 91 routes of the walks suite, crosses a wall, and routes use the stairs between floors. **RQ2**

is answered yes for the five class groups: each class links to a room, and the next class is shown with directions, although only five room codes could be linked. **RQ3** is answered yes at emulated sizes: the whole route is visible beside the panel on a phone, and every button meets 44 px, but no real phone was used. **RQ4** is answered partly: floor changes and routes are well within 2 seconds, but a first visit on the slower profile is not.

Chapter 5

Discussion

5.1 Critical Discussion

The main result is that a free, browser-based application can give wall-aware routes across two floors of a building and the outdoor campus, using only satellite traces, a few measured walks and existing drawings. The strength of the evidence is uneven. Route validity is strong, because it is checked automatically on every sampled position of over 1 500 routes. Usability evidence is weaker: it rests on automated checks of sizes and visibility at emulated phone sizes, not on observing students. Positional accuracy is limited by the drawings and footprint: a median error of 1.6 m is enough to tell rooms apart, but individual rooms on the south-east wing may be several metres out.

The first-visit time is the main performance weakness. Almost all of the 2.5 MB transferred are the two drawings (2.0 MB). After the first visit the service worker serves them from the cache, so later visits do not depend on the network; the 3.4 s figure therefore applies to a student's first use.

5.2 Comparison with Previous Studies

Like Find 4 Me (Helmi et al., 2022), BAC Maps targets new students and visitors, but it adds outdoor routes from the gate and a link to the timetable, and it publishes measured test results. Unlike the application of Patel et al. (2025), which combines Unity indoors and Mapbox outdoors, one engine plans the whole route, so there is no hand-over at a building's door. The finding of Ausmeier et al. (2012) that plan annotation is a practical way to capture building data is consistent with this project, where the drawings supplied the room layout and the measured walks served as checks. Compared with commercial platforms (Mappedin, 2026; Mapbox, 2026), BAC Maps lacks their device support and tooling, but needs no licence or prepared data.

5.3 Explanation of Unexpected Results

- **Routes hidden behind the panel.** Centring a route on the screen seemed natural, but the panel covers the centre on both desktop and phone. This led to C3 and to measuring the visible share of each route.
- **No routes in a background tab.** Pathfinder started its work on the next screen redraw, which browsers do not run in hidden tabs; the automated tests, which run in the background, exposed this.
- **Missing class groups.** Databases created before groups existed kept working but showed no timetable, which only appeared when an older database was reused.
- **Older test failures.** The four failures in the Pathfinder demonstration page's suite also occur before the merge. They test behaviour of an older map (for example

tilting it or dragging it off the campus) that the drawing-based map no longer has, so the tests, not the map, are out of date.

Chapter 6

Conclusion and Recommendations

6.1 Summary of Key Findings

- **O1, met.** The map shows the campus and both floors of the Main Building; all 40 rooms are searchable (FR-01 to FR-06).
- **O2, met for mapped rooms.** Routes reach every room with a door; no checked route crosses a wall. Only Office 3 is unreachable, and the application says why (FR-07 to FR-14).
- **O3, partly met.** Group timetables, own classes, saved places and the next class work for the five class groups, but sign-in uses demonstration accounts only (FR-15 to FR-19).
- **O4, partly met.** Floor changes and routes are well within 2 seconds, and every button meets 44 px, but the first map display on the slower profile takes 3.4 seconds (NFR-01, NFR-07).
- **O5, met for the demonstration server.** Every security check passed (NFR-04).

6.2 Contributions to Knowledge

The project shows a low-cost method for campus navigation where no geographic database exists: tracing over satellite imagery, aligning existing drawings with a thin-plate spline, and confirming key paths with measured walks. It also shows that an admissible A* estimate scaled by the cheapest surface lets routes prefer verified paths without losing optimality.

6.3 Practical Implications

BAC could offer the application to new students during orientation once it is hosted over HTTPS and connected to college accounts. Updating the map requires editing data files and rerunning the build scripts, not changing code, so it can be maintained by staff with modest technical skills.

6.4 Limitations

Sign-in is a demonstration with dummy accounts. Room positions are approximate, and interiors exist only for the Main Building. No real phone, field test or user study was carried out, contrast was not measured, and the server was not load-tested. The camera-based door recognition, the arrival alert and room capacity were not implemented. Only five timetable room codes are linked to places.

6.5 Recommendations for Future Research

1. Test with new students on campus with real phones, measuring time to reach a room with and without the application.

2. Connect sign-in to the college's accounts and host the site over HTTPS, then test installation and offline use on Android and iOS.

3. Reduce the first-visit time, for example by loading a smaller drawing first and the full drawing afterwards.

4. Survey room doors and positions on site, starting with Office 3 and the south-east wing, and add interiors for other buildings.

5. Revisit the camera feature as door signs with QR codes, which need no photo database, before attempting image recognition.

6. Update the Pathfinder demonstration page's tests to the drawing-based map.

References

Ausmeier, B., Campbell, T. and Berman, S. (2012) 'Indoor navigation using a mobile phone', in *2012 African Conference for Software Engineering and Applied Computing*. Gaborone, Botswana: IEEE, pp. 109–115. doi:10.1109/ACSEAC.2012.26.

Bookstein, F.L. (1989) 'Principal warps: thin-plate splines and the decomposition of deformations', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(6), pp. 567–585. doi:10.1109/34.24792.

Guntupally, K., Devarakonda, R. and Kehoe, K. (2018) 'Spring Boot based REST API to improve data quality report generation for big scientific data: ARM data center example', in *2018 IEEE International Conference on Big Data*. Seattle, WA: IEEE, pp. 5328–5329. doi:10.1109/BigData.2018.8621924.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107. doi:10.1109/TSSC.1968.300136.

Helmi, R.A.A., Ravichandran, H.A.-P., Jamal, A. and Mohammed, M.N. (2022) 'Design and development of indoor campus navigation application', in *2022 IEEE 10th Conference on Systems, Process & Control (ICSPC)*. Malacca, Malaysia: IEEE, pp. 77–82. doi:10.1109/ICSPC55597.2022.10001791.

Hunt, R. (2026) *Detailed isometric illustration and illustrated maps by Rod Hunt*. Available at: `https://www.rodhunt.com/` (Accessed: 24 October 2026).

Mapbox (2026) *Mapbox GL JS guides*. Available at: `https://docs.mapbox.com/mapbox-gl-js/guides/` (Accessed: 24 October 2026).

Mappedin (2026) *Mappedin developer portal*. Available at: `https://developer.mappedin.com/` (Accessed: 24 October 2026).

Moriarty, K. (ed.), Kaliski, B. and Rusch, A. (2017) *PKCS #5: password-based cryptography specification version 2.1*. RFC 8018. Internet Engineering Task Force. doi:10.17487/RFC8018.

MyTimetable (2026) *Personalised timetable integration*. Available at: `https://mytimetable.net/` (Accessed: 24 October 2026).

Patel, R., Patel, R. and Dwivedi, J.N. (2025) 'Campus navigation and augmented reality guided mobile application', in *2025 3rd International Conference on Communication, Security, and Artificial Intelligence (ICCSAI)*. Greater Noida, India: IEEE, pp. 1802–1807. doi:10.1109/ICCSAI64074.2025.11064653.

University of British Columbia (2026) *Wayfinding at UBC, Vancouver campus*. Available at: `https://maps.ubc.ca/` (Accessed: 24 October 2026).

W3C (2024) *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 12 December. Available at: `https://www.w3.org/TR/WCAG22/` (Accessed: 24 October 2026).

W3C (2026a) *Geolocation*. W3C Candidate Recommendation Snapshot, 19 March. Available at: `https://www.w3.org/TR/geolocation/` (Accessed: 24 October 2026).

W3C (2026b) *Service Workers*. W3C Candidate Recommendation Draft, 17 Septem-

ber. Available at: https://www.w3.org/TR/service-workers/ (Accessed: 24 October 2026).

Appendix A

Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
BAC	Botswana Accountancy College
BSBS	the project's original name (BSBS Maps)
CAF	cafeteria
DOM	Document Object Model
FR	Functional Requirement
GPS	Global Positioning System
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
KML	Keyhole Markup Language
MoSCoW	Must, Should, Could, Won't
NFR	Non-Functional Requirement
PBKDF2	Password-Based Key Derivation Function 2
PWA	Progressive Web App
SCIS	School of Computing and Information Systems [**INSERT confirm the full name**]
SQL	Structured Query Language
TPS	thin-plate spline
UI	user interface
W3C	World Wide Web Consortium
WCAG	Web Content Accessibility Guidelines

Appendix B

Requirement Cards

Table B.1: Acceptance criteria for the revised functional requirements

ID	Requirement	Acceptance criteria	NFRs
FR-01	Interactive campus map: pan, zoom, reset	Drawings shown on desktop and phone; buildings distinguishable; reset shows the whole campus	01, 03, 07
FR-02	Floor switching	All, Ground, First change the view without a reload	01
FR-03	Search	Typing part of a name lists matching buildings, rooms and facilities	01
FR-04	Quick picks and directory	Each chip and category lists only its places	07
FR-05	Map tap selects	Tapping a building opens its card	03
FR-06	Place card	Shows category, building and floor; Directions, Save and Show on map work	07
FR-07	Gate-to-room route	A route from the Main Gate to any room with a door is drawn	01
FR-08	Multi-floor route and steps	Route uses mapped doors and stairs; no sample inside a wall; steps grouped by floor	01
FR-09	Alternative route	A second, different route is offered when one exists	01
FR-10	Change start or destination	Picking a new start or end recalculates the route	01
FR-11	Start from the user's position	On campus with permission, the route starts at the position; otherwise at the Main Gate with a message	04
FR-12	Walk preview and step pin	Walk it moves a marker along the route; a tapped step is pinned	01
FR-13	No-route explanation	An unreachable place shows the reason instead of a route	07

ID	Requirement	Acceptance criteria	NFRs
FR-14	Route clear of the panel	At least 90% of the route visible on desktop and phone	03, 07
FR-15	Sign in and out	Valid account signs in; wrong password refused; sign-out ends the session	04
FR-16	Group timetable	The student's group classes shown by day, with loading, error and empty states	04
FR-17	Own classes	Add, edit and delete; invalid times refused	04
FR-18	Next class	The next class is shown with a working Directions button	01
FR-19	Saved places	Save and unsave; list shown in the profile	04
FR-20	Staff map corrections	Corrections change routes without code changes	06

Appendix C

Test Commands and Run Instructions

The application is run from the repository root:

```
python server/server.py     (then open 127.0.0.1:8765/site/)
cd frontend && npm install && npm run build   (rebuilds site/)
```

The suites in Table 4.2 are run with:

```
node tests/nav.test.js   (and the other four .test.js files)
python tests/e2e_site.py --edge
python tests/e2e_profile.py --edge
python tests/e2e_floors.py   (run with Edge here)
```

The raw results of 24 October 2026 are kept in `report/evidence/test-run.txt` and `report/evidence/measurements.json`. The demonstration account details are in `server/seed.py` and are not reproduced here.